AI's Technology Stack Recommendation

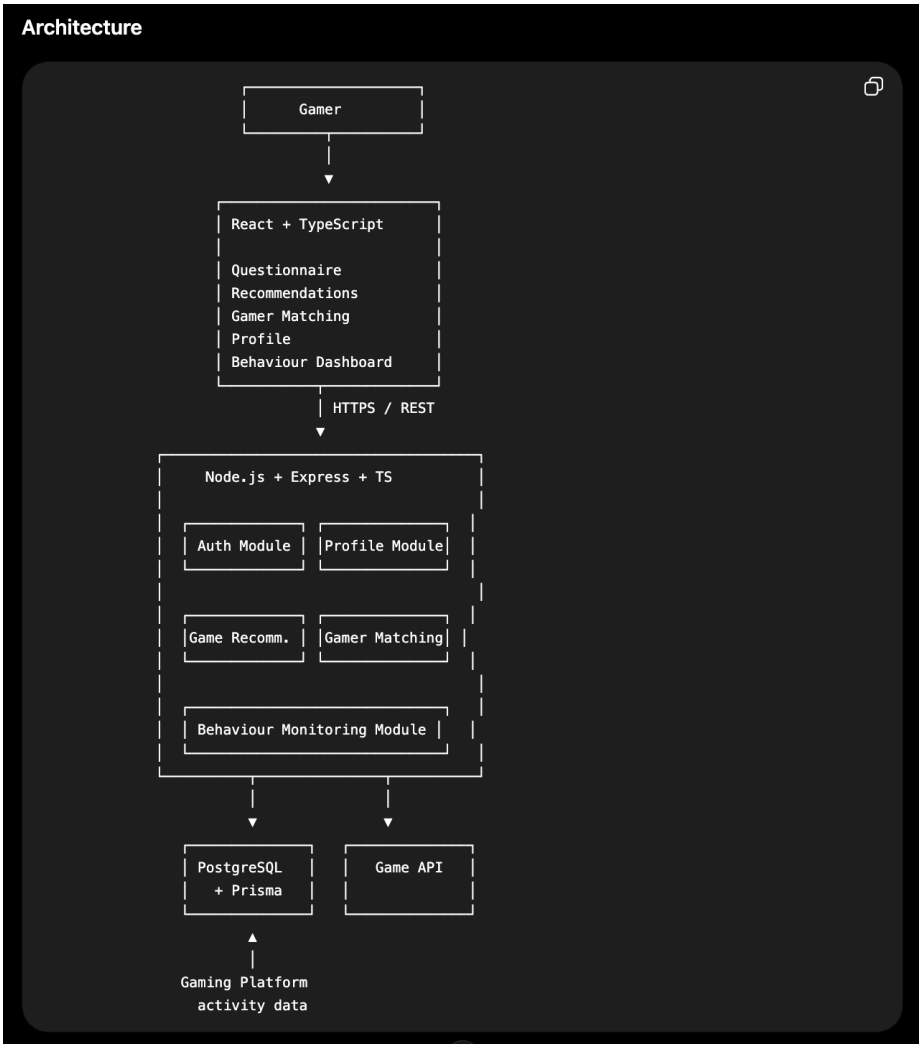
Description of uploaded materials:

- **Vibecheck's SRS document**
- **UI Mock ups**
- **Vibecheck's Use Case diagram**

Prompt: Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team. Structure the stack towards developing a dynamic website with a realtime database.

AI's Recommendation: Based on your use-case diagram and the requirements in the Gaming Project SRS, I would keep the architecture deliberately simple: a modular monolith, not microservices. Your system needs questionnaires/profile generation, gamer matching, game recommendations, gaming-behaviour monitoring, authentication, and communication with an external Game API. The SRS also requires responsive desktop/mobile UI, separable modules, password hashing, privacy controls, and only about 30 concurrent users, so you do not need a complicated distributed architecture.

Layer	Technology	Why it fits your project
Frontend	React + TypeScript + Vite	Beginner-friendly, huge ecosystem, good for questionnaire/profile/recommendation interfaces
Styling	Tailwind CSS	Fast to build responsive UI; helps satisfy desktop/tablet/mobile requirement
Backend	Node.js + Express + TypeScript	Same language as frontend; much easier for an entry-level team
API style	REST API	Simple and sufficient for your use cases
Database	PostgreSQL	Excellent fit for users, profiles, matches, games and activity records
ORM	Prisma	Much easier database access than writing raw SQL everywhere
Authentication	JWT + bcrypt	Straightforward login implementation; bcrypt satisfies hashed-password requirement
External game data	Game API through backend	Matches the `Game API` actor in your diagram
Recommendation / matching	TypeScript modules initially	No need for ML infrastructure; your current requirements only require profile-based algorithms
Testing	Vitest/Jest + React Testing Library + Supertest	Covers frontend, backend and API testing
API testing	Postman	Easy for a student team to test endpoints together
Version control	Git + GitHub	Collaboration, issues, branches and pull requests
CI	GitHub Actions	Automatically run tests when code is pushed
Frontend hosting	Vercel	Very easy React deployment
Backend hosting	Render / Railway	Easy Node deployment without infrastructure management
Database hosting	Supabase / Neon PostgreSQL	Managed PostgreSQL; little DevOps work



This also lines up nicely with the SRS requirement that core functionalities should be implemented as separable modules.

Why I would choose JavaScript/TypeScript throughout

For your SC2006 team, I would strongly favour:

React → TypeScript → Express → Prisma → PostgreSQL

rather than something like:

React → Java Spring Boot → Python ML service → MongoDB → Redis → Docker/Kubernetes

The second architecture may look more impressive, but it creates unnecessary integration work.

Opinion on AI's Recommendation

AI had separate recommendations for Database and Authentication, PostgreSQL, and JWT (JSON Web Token) and Bcrypt. However, it would take a lot of time to integrate both into the project, it would also be a hassle to host the database. User privacy may also be an issue if we do not properly implement it. Additionally, PostgreSQL is not a realtime database, this would affect the user experience as we cannot update information in real time.

A good alternative that we would be using is Firebase. Firebase has authentication and realtime database features. Hence, it would be able to handle both for us.